

Федеральное государственное автономное образовательное учреждение
высшего образования

Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 Программная инженерия

Дисциплина «Распределённые системы хранения данных»

Отчёт

по лабораторной работе №3

Вариант:

Студент:

Чусовлянов Максим Сергеевич
Садовой Григорий Владимирович
Группа Р3307

Преподаватель:

Максимов Андрей Николаевич

г. Санкт-Петербург, 2026 г.

Оглавление

Задание	3
Архитектура стенда	4
Этап 1. Конфигурация	5
Подготовка проекта	5
Проверка ролей серверов	6
Проверка PgBouncer и тестовой базы	6
Этап 2. Симуляция и обработка сбоя	8
2.1 Подготовка	8
2.2 Сбой	9
2.3 Обработка сбоя	10
Этап 3. Восстановление	12
Откат действия из этапа 2.2	12
Актуализация базы на pg1	12
Проверка сетевого потока репликации	13
Возврат исходной конфигурации	13
Финальная проверка	15
Архитектура	17
PgBouncer aliases	17
Ansible templates	17
Пользователи	17
Состояние базы до сбоя и после восстановления	17
Фрагменты Ansible-шаблонов	19
Шаблон postgresql-primary.conf.j2	19
Шаблон postgresql-standby.conf.j2	19
Шаблон pg_hba-primary.conf.j2	20
Шаблон pg_hba-standby.conf.j2	20
Шаблон pgbouncer.ini.j2	20
Шаблон userlist.txt.j2	21
Скрипт проверки доступности и переключения	22
Systemd service и timer	23
Вывод	25

Задание

Цель работы — ознакомиться с методами и средствами построения отказоустойчивых решений на базе СУБД PostgreSQL; получить практические навыки восстановления работы системы после отказа.

В работе необходимо развернуть PostgreSQL на двух узлах в режиме балансировки нагрузки с использованием PgBouncer, продемонстрировать работу с базой, симулировать отказ основного узла переполнением раздела с PGDATA, выполнить failover на резервный сервер и восстановить исходную конфигурацию.

Архитектура стенда

Для выполнения работы используются три одинаковые виртуальные машины Ubuntu 22.04.

Узел	IP	Назначение
pg1	203.0.113.10	основной PostgreSQL до сбоя
pg2	203.0.113.11	резервный PostgreSQL до сбоя
client	203.0.113.12	отдельная машина для PgBouncer, Ansible и клиентских подключений

До сбоя схема имеет вид:

```
client -> PgBouncer -> pg1 primary -> pg2 standby
```

После отказа основного узла автоматический скрипт переводит pg2 в primary и переключает PgBouncer на него:

```
client -> PgBouncer -> pg2 primary
```

На этапе восстановления вручную возвращаем исходную конфигурацию:

```
client -> PgBouncer -> pg1 primary -> pg2 standby
```

Этап 1. Конфигурация

Подготовка проекта

Ansible запускается с машины `client`. Проект копируется на `client`, после чего проверяется доступность всех ВМ.

```
ssh root@203.0.113.12
cd /root/lab3-postgres-ha
ansible all -i inventory.ini -m ping

root@client-host:~/lab3# ansible -m ping all -k -K
SSH password:
BECOME password[defaults to SSH password]:
[WARNING]: Found both group and host with same name: client
pg1 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
client | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
pg2 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
root@client-host:~/lab3#
```

Основная настройка стенда выполняется `playbook`'ом:

```
ansible-playbook -i inventory.ini site.yml
```

В результате `playbook` устанавливает PostgreSQL на `pg1` и `pg2`, настраивает репликацию, создаёт тестовую базу, устанавливает PgBouncer на `client`, а также размещает скрипт автоматического failover.

TASK [Показываю роли Postgres]

```
ok: [client] =>
    msg: primary recovery=n/a, standby recovery=n/a
ok: [pg1] =>
    msg: primary recovery=f, standby recovery=n/a
ok: [pg2] =>
    msg: primary recovery=n/a, standby recovery=t
```

PLAY RECAP

client		: ok=28	changed=2	unreachable=0	failed=0
skipped=2	rescued=0	ignored=0			
pg1		: ok=28	changed=4	unreachable=0	failed=0
skipped=2	rescued=0	ignored=0			

```
pg2                                : ok=23   changed=5   unreachable=0   failed=0
skipped=2   rescued=0   ignored=0
```

Проверка ролей серверов

На pg1 проверяем, что сервер является primary:

```
ssh root@203.0.113.10
sudo -u postgres psql -d labdb -c "SELECT pg_is_in_recovery();"
```

Ожидаемый результат: f.

На pg2 проверяем, что сервер является standby:

```
ssh root@203.0.113.11
sudo -u postgres psql -d labdb -c "SELECT pg_is_in_recovery();"
```

Ожидаемый результат: t.

На pg1 проверяем потоковую репликацию:

```
sudo -u postgres psql -d labdb -c "SELECT client_addr, state, sync_state FROM
pg_stat_replication;"
```

Проверка PgBouncer и тестовой базы

Подключение к СУБД выполняется с отдельной машины client через PgBouncer. В PgBouncer настроены три входа: labdb и labdb_rw ведут на основной сервер, labdb_ro ведёт на резервный сервер для чтения.

```
ssh root@203.0.113.12
grep "labdb" /etc/pgbouncer/pgbouncer.ini
psql -h 127.0.0.1 -p 6432 -U appuser -d labdb_rw
```

Проверяем наполнение базы:

```
SELECT * FROM clients_lab;
SELECT * FROM operations_lab;
```

Проверяем транзакцию записи:

```
BEGIN;
INSERT INTO clients_lab(name, balance) VALUES ('BeforeFailover', 1000);
INSERT INTO operations_lab(client_id, amount, operation_type)
VALUES (currval('clients_lab_id_seq'), 1000, 'before_failover');
COMMIT;
```

Проверяем чтение с реплики через PgBouncer:

```
psql -h 127.0.0.1 -p 6432 -U appuser -d labdb_ro
SELECT pg_is_in_recovery();
SELECT * FROM clients_lab;
```

Ожидаемый результат: чтение работает, pg_is_in_recovery() возвращает t. Если выполнить INSERT, реплика вернёт ошибку, потому что standby доступен только на чтение.

Один из фактических выводов проверки записи через labdb_rw и чтения через labdb_ro:

```
labdb_rw=> SELECT * FROM clients_lab;
```

id	name	balance	created_at
1	Ivan	1000	2026-06-04 09:42:29.910633+00
2	Maria	1500	2026-06-04 09:42:29.911326+00

(2 rows)

```
labdb_rw=> BEGIN;
```

```
BEGIN
```

```
labdb_rw=> INSERT INTO clients_lab(name, balance) VALUES ('BeforeFailover',  
1000);
```

```
INSERT 0 1
```

```
labdb_rw=> INSERT INTO operations_lab(client_id, amount, operation_type) VALUES  
(currval('clients_lab_id_seq'), 1000, 'before_failover');
```

```
INSERT 0 1
```

```
labdb_rw=> COMMIT;
```

```
COMMIT
```

```
labdb_rw=> SELECT * FROM clients_lab;
```

id	name	balance	created_at
1	Ivan	1000	2026-06-04 09:42:29.910633+00
2	Maria	1500	2026-06-04 09:42:29.911326+00
3	BeforeFailover	1000	2026-06-04 09:44:44.184648+00

(3 rows)

```
labdb_ro=> SELECT pg_is_in_recovery();
```

pg_is_in_recovery
t

(1 row)

```
labdb_ro=> SELECT * FROM clients_lab;
```

id	name	balance	created_at
1	Ivan	1000	2026-06-04 09:42:29.910633+00
2	Maria	1500	2026-06-04 09:42:29.911326+00
3	BeforeFailover	1000	2026-06-04 09:44:44.184648+00

(3 rows)

```
labdb_ro=> INSERT INTO clients_lab(name, balance) VALUES ('BadReplicaWrite', 1);  
ERROR:  cannot execute INSERT in a read-only transaction
```

На pg2 проверяем, что данные синхронизировались на резервный сервер:

```
ssh root@203.0.113.11
```

```
sudo -u postgres psql -d labdb -c "SELECT * FROM clients_lab;"
```

Этап 2. Симуляция и обработка сбоя

2.1 Подготовка

На client открываем несколько терминалов и подключаемся к базе через PgBouncer:

```
psql -h 127.0.0.1 -p 6432 -U appuser -d labdb_rw
```

Для чтения с реплики можно открыть отдельную сессию:

```
psql -h 127.0.0.1 -p 6432 -U appuser -d labdb_ro
```

В одной сессии выполняем чтение:

```
SELECT * FROM clients_lab;  
SELECT * FROM operations_lab;
```

В другой сессии выполняем транзакцию записи:

```
BEGIN;  
INSERT INTO clients_lab(name, balance) VALUES ('SessionBeforeFail', 1500);  
INSERT INTO operations_lab(client_id, amount, operation_type)  
VALUES (currval('clients_lab_id_seq'), 1500, 'session_before_fail');  
COMMIT;
```

Фактические выводы двух клиентских сессий:

```
labdb_rw=> BEGIN;  
BEGIN  
labdb_rw=> INSERT INTO clients_lab(name, balance) VALUES ('SessionOneClient',  
1111);  
INSERT 0 1  
labdb_rw=> INSERT INTO operations_lab(client_id, amount, operation_type) VALUES  
(currval('clients_lab_id_seq'), 1111, 'session_one_write');  
INSERT 0 1  
labdb_rw=> COMMIT;  
COMMIT
```

id	name	balance	created_at
42	SessionOneClient	1111	2026-06-04 10:14:06.036186+00
41	TcpdumpReplicationProof	8888	2026-06-04 10:11:57.976054+00
40	ReplicationProof	7777	2026-06-04 10:09:09.067596+00
39	RecoverySyncCheck	2500	2026-06-04 10:07:57.042234+00
38	AfterRecovery	3000	2026-06-04 10:05:57.186191+00

(5 rows)

id	client_id	amount	operation_type	created_at
38	42	1111	session_one_write	2026-06-04 10:14:06.036186+00
37	38	3000	after_recovery	2026-06-04 10:05:57.186191+00
36	37	2000	after_failover	2026-06-04 09:57:00.42334+00
3	3	1000	before_failover	2026-06-04 09:44:44.184648+00
2	2	200	initial	2026-06-04 09:42:29.912284+00

(5 rows)

```
labdb_rw=> BEGIN;
BEGIN
labdb_rw=> INSERT INTO clients_lab(name, balance) VALUES ('SessionTwoClient',
2222);
INSERT 0 1
labdb_rw=> INSERT INTO operations_lab(client_id, amount, operation_type) VALUES
(currval('clients_lab_id_seq'), 2222, 'session_two_write');
INSERT 0 1
labdb_rw=> COMMIT;
COMMIT
```

id	name	balance	created_at
43	SessionTwoClient	2222	2026-06-04 10:14:41.968272+00
42	SessionOneClient	1111	2026-06-04 10:14:06.036186+00
41	TcpdumpReplicationProof	8888	2026-06-04 10:11:57.976054+00
40	ReplicationProof	7777	2026-06-04 10:09:09.067596+00
39	RecoverySyncCheck	2500	2026-06-04 10:07:57.042234+00

(5 rows)

id	client_id	amount	operation_type	created_at
39	43	2222	session_two_write	2026-06-04 10:14:41.968272+00
38	42	1111	session_one_write	2026-06-04 10:14:06.036186+00
37	38	3000	after_recovery	2026-06-04 10:05:57.186191+00
36	37	2000	after_failover	2026-06-04 09:57:00.42334+00
3	3	1000	before_failover	2026-06-04 09:44:44.184648+00

(5 rows)

2.2 Сбой

На основном узле pg1 заполняем раздел с PGDATA мусорными файлами:

```
ssh root@203.0.113.10
sudo /usr/local/bin/fill_pgdata_disk.sh
```

Проверяем заполнение диска:

```
df -h /var/lib/postgresql/14/main
```

Фактический результат заполнения:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/vda2	79G	74G	1.5G	99%	/

Симуляция переполнения завершена. Папка с мусорными файлами: /var/lib/postgresql/14/main/trash

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/vda2	79G	75G	266M	100%	/

После этого была выполнена тяжёлая запись через PgBouncer:

```
PGPASSWORD='<APP_PASSWORD>' psql -h 127.0.0.1 -p 6432 -U appuser -d labdb_rw -c  
"CREATE TABLE disk_fill_test AS SELECT generate_series(1, 10000000) AS id,  
repeat('x', 1000) AS payload;"
```

Фактическая ошибка PostgreSQL:

```
PANIC: could not write to file "pg_wal/xlogtemp.13938": No space left on device  
CONTEXT: writing block 16267 of relation base/16386/16414  
FATAL: server conn crashed?  
server closed the connection unexpectedly  
This probably means the server terminated abnormally  
before or while processing the request.  
connection to server was lost
```

2.3 Обработка сбоя

На pg1 смотрим логи PostgreSQL и находим сообщения об ошибках:

```
journalctl -u postgresql -n 100
```

Также можно отфильтровать сообщения:

```
grep -i "no space\|could not\|error\|fatal\|panic" /var/log/postgresql/  
postgresql-*.log
```

Failover выполняется автоматически. На client смотрим лог auto-failover:

```
sudo tail -f /var/log/pg_auto_failover.log
```

Фактический вывод лога failover:

```
2026-06-04 09:51:49 Проверяем основной узел 203.0.113.10...  
2026-06-04 09:51:49 Основной узел 203.0.113.10 доступен.  
2026-06-04 09:51:55 Проверяем основной узел 203.0.113.10...  
2026-06-04 09:51:55 Основной узел 203.0.113.10 недоступен. Проверяем резервный  
узел 203.0.113.11...  
2026-06-04 09:51:55 Резервный узел 203.0.113.11 доступен. Выполняем promote  
через SSH...  
waiting for server to promote.... done  
server promoted  
2026-06-04 09:52:00 Резервный узел 203.0.113.11 успешно повышен до primary.  
Переключаем PgBouncer.  
2026-06-04 09:52:00 PgBouncer переключён на 203.0.113.11 и перезапущен.  
2026-06-04 09:52:00 Текущий primary уже 203.0.113.11. Действие не требуется.
```

Проверяем, что pg2 стал primary:

```
ssh root@203.0.113.11 "sudo -u postgres psql -d labdb -c 'SELECT  
pg_is_in_recovery();'"
```

Результат проверки:

```
could not change directory to "/root": Permission denied  
pg_is_in_recovery  
-----  
f  
(1 row)
```

На client проверяем, что PgBouncer теперь смотрит на pg2:

```
grep "labdb" /etc/pgbouncer/pgbouncer.ini
```

Результат:

```
labdb = host=203.0.113.11 port=5432 dbname=labdb
labdb_rw = host=203.0.113.11 port=5432 dbname=labdb
labdb_ro = host=203.0.113.11 port=5432 dbname=labdb
```

После переключения снова выполняем запись через PgBouncer:

```
PGPASSWORD='<APP_PASSWORD>' psql -h 127.0.0.1 -p 6432 -U appuser -d labdb_rw -c
"BEGIN; INSERT INTO clients_lab(name, balance) VALUES ('AfterFailover', 2000);
INSERT INTO operations_lab(client_id, amount, operation_type) VALUES
(currval('clients_lab_id_seq'), 2000, 'after_failover'); COMMIT; SELECT * FROM
clients_lab; SELECT * FROM operations_lab;"
```

Результат показывает, что после failover запись выполняется уже на новом primary pg2:

id	client_id	amount	operation_type	created_at
1	1	100	initial	2026-06-04 09:42:29.911634+00
2	2	200	initial	2026-06-04 09:42:29.912284+00
3	3	1000	before_failover	2026-06-04 09:44:44.184648+00
36	37	2000	after_failover	2026-06-04 09:57:00.42334+00

(4 rows)

Этап 3. Восстановление

После failover актуальным primary является pg2. Старый основной узел pg1 нельзя просто включать обратно как primary, так как на pg2 уже появились новые данные. Поэтому сначала восстанавливаем pg1 как standby от pg2, затем вручную возвращаем исходные роли.

Откат действия из этапа 2.2

На client останавливаем автоматический failover:

```
sudo systemctl stop pg-auto-failover.timer
```

На pg1 удаляем мусорные файлы:

```
ssh root@203.0.113.10
sudo rm -rf /var/lib/postgresql/14/main/trash
sudo rm -f /var/lib/postgresql/14/main/trashfile*
df -h /var/lib/postgresql/14/main
```

Актуализация базы на pg1

На pg1 пересоздаём каталог данных из актуального primary pg2:

```
sudo systemctl stop postgresql
sudo rm -rf /var/lib/postgresql/14/main/*
sudo -u postgres bash
export PGPASSWORD='<REPLICATION_PASSWORD>'
pg_basebackup -h 203.0.113.11 -D /var/lib/postgresql/14/main -U replicator -P -R -X stream
exit
sudo systemctl start postgresql
sudo -u postgres psql -d labdb -c "SELECT pg_is_in_recovery();"

```

Ожидаемый результат на pg1: t.

Фактическая проверка актуализации pg1 после pg_basebackup от pg2:

```
root@client-host:~# ssh root@203.0.113.10 "sudo -u postgres psql -d labdb -c
'SELECT pg_is_in_recovery();"
pg_is_in_recovery
-----
t
(1 row)

```

```
root@client-host:~# ssh root@203.0.113.10 "sudo -u postgres psql -d labdb -c
\"SELECT * FROM clients_lab WHERE name = 'AfterFailover';\""
 id |      name      | balance |      created_at
----+-----+-----+-----
 37 | AfterFailover |    2000 | 2026-06-04 09:57:00.42334+00
(1 row)

```

```
root@client-host:~# ssh root@203.0.113.11 "sudo -u postgres psql -d labdb -c
'SELECT client_addr, state, sync_state FROM pg_stat_replication;'"

```

```

client_addr | state | sync_state
-----+-----+-----
203.0.113.10 | streaming | async
(1 row)

```

```

root@client-host:~# PGPASSWORD='<APP_PASSWORD>' psql -h 127.0.0.1 -p 6432 -U
appuser -d labdb_rw -c "INSERT INTO clients_lab(name, balance) VALUES
('ReplicationProof', 7777);"
INSERT 0 1

```

```

root@client-host:~# ssh root@203.0.113.10 "sudo -u postgres psql -d labdb -c
\SELECT * FROM clients_lab WHERE name = 'ReplicationProof';\"

```

id	name	balance	created_at
40	ReplicationProof	7777	2026-06-04 10:09:09.067596+00

```

(1 row)

```

Проверка сетевого потока репликации

Для дополнительной проверки на pg2 был запущен tcpdump по порту PostgreSQL:

```
tcpdump -i any -nn host 203.0.113.10 and port 5432
```

Фрагмент фактического вывода:

```

10:14:06.069467 ens3 Out IP 203.0.113.11.5432 > 203.0.113.10.59094: Flags [P.],
seq 661:2675, ack 975, win 509, options [nop,nop,TS val 1930422790 ecr
4261116046], length 2014
10:14:06.079801 ens3 In IP 203.0.113.10.59094 > 203.0.113.11.5432: Flags [.],
ack 2675, win 497, options [nop,nop,TS val 4261120184 ecr 1930422790], length 0
10:14:06.080047 ens3 In IP 203.0.113.10.59094 > 203.0.113.11.5432: Flags [P.],
seq 975:1014, ack 2675, win 501, options [nop,nop,TS val 4261120184 ecr
1930422790], length 39
10:14:42.001946 ens3 Out IP 203.0.113.11.5432 > 203.0.113.10.59094: Flags [P.],
seq 2761:3207, ack 1287, win 509, options [nop,nop,TS val 1930458722 ecr
4261148304], length 446
10:14:42.012367 ens3 In IP 203.0.113.10.59094 > 203.0.113.11.5432: Flags [P.],
seq 1287:1326, ack 3207, win 501, options [nop,nop,TS val 4261156117 ecr
1930458722], length 39

```

В это время через PgBouncer была выполнена запись:

```

root@client-host:~# PGPASSWORD='<APP_PASSWORD>' psql -h 127.0.0.1 -p 6432 -U
appuser -d labdb_rw -c "INSERT INTO clients_lab(name, balance) VALUES
('TcpdumpReplicationProof', 8888);"
INSERT 0 1

```

Возврат исходной конфигурации

На client временно останавливаем PgBouncer, чтобы во время переключения не было новых записей:

```
sudo systemctl stop pgbouncer
```

На pg1 повышаем сервер обратно до primary:

```
sudo -u postgres /usr/lib/postgresql/14/bin/pg_ctl promote -D /var/lib/postgresql/14/main
sudo -u postgres psql -d labdb -c "SELECT pg_is_in_recovery();"
```

Ожидаемый результат на pg1: f.

На pg2 пересоздаём standby от pg1:

```
ssh root@203.0.113.11
sudo systemctl stop postgresql
sudo rm -rf /var/lib/postgresql/14/main/*
sudo -u postgres bash
export PGPASSWORD='<REPLICATION_PASSWORD>'
pg_basebackup -h 203.0.113.10 -D /var/lib/postgresql/14/main -U replicator -P -R -X stream
exit
sudo systemctl start postgresql
sudo -u postgres psql -d labdb -c "SELECT pg_is_in_recovery();"
```

Ожидаемый результат на pg2: t.

На client вручную возвращаем PgBouncer на pg1. Открываем конфигурационный файл:

```
sudo nano /etc/pgbouncer/pgbouncer.ini
```

Скриншот ручного изменения конфигурации PgBouncer:



Рис. 1. Ручное возвращение PgBouncer на pg1

В секции [databases] возвращаем строки к исходному состоянию:

```
labdb = host=203.0.113.10 port=5432 dbname=labdb
labdb_rw = host=203.0.113.10 port=5432 dbname=labdb
labdb_ro = host=203.0.113.11 port=5432 dbname=labdb
```

Сохраняем файл и запускаем сервисы:

```
echo '203.0.113.10' | sudo tee /var/lib/pgbouncer/current_primary
sudo systemctl start pgbouncer
sudo systemctl start pg-auto-failover.timer
```

Финальная проверка

Фактический вывод возврата к исходной конфигурации:

```
root@client-host:~# ssh root@203.0.113.10
root@client-host:~# sudo -u postgres /usr/lib/postgresql/14/bin/pg_ctl promote -
D /var/lib/postgresql/14/main
waiting for server to promote.... done
server promoted
root@client-host:~# sudo -u postgres psql -d labdb -c "SELECT
pg_is_in_recovery();"
 pg_is_in_recovery
-----
 f
(1 row)

root@client-host:~# ssh root@203.0.113.11
root@client-host:~# systemctl stop postgresql
root@client-host:~# rm -rf /var/lib/postgresql/14/main/*
root@client-host:~# sudo -u postgres pg_basebackup -h 203.0.113.10 -D /var/lib/
postgresql/14/main -U replicator -P -R
Password:
104080/104080 kB (100%), 1/1 tablespace
root@client-host:~# systemctl start postgresql
root@client-host:~# sudo -u postgres psql -d labdb -c "SELECT
pg_is_in_recovery();"
 pg_is_in_recovery
-----
 t
(1 row)

root@client-host:~# ssh root@203.0.113.10 "sudo -u postgres psql -d labdb -c
'SELECT pg_is_in_recovery();'"
 pg_is_in_recovery
-----
 f
(1 row)

root@client-host:~# ssh root@203.0.113.11 "sudo -u postgres psql -d labdb -c
'SELECT pg_is_in_recovery();'"
 pg_is_in_recovery
-----
 t
(1 row)

root@client-host:~# grep "labdb" /etc/pgbouncer/pgbouncer.ini
```

```
labdb = host=203.0.113.10 port=5432 dbname=labdb
labdb_rw = host=203.0.113.10 port=5432 dbname=labdb
labdb_ro = host=203.0.113.11 port=5432 dbname=labdb
```

На client подключаемся через PgBouncer:

```
psql -h 127.0.0.1 -p 6432 -U appuser -d labdb
```

Проверяем, что данные, добавленные после failover, сохранились:

```
SELECT * FROM clients_lab;
SELECT * FROM operations_lab;
```

Выполняем новую транзакцию после восстановления:

```
BEGIN;
INSERT INTO clients_lab(name, balance) VALUES ('AfterRecovery', 3000);
INSERT INTO operations_lab(client_id, amount, operation_type)
VALUES (currval('clients_lab_id_seq'), 3000, 'after_recovery');
COMMIT;
```

Фактический результат финальной записи:

```
root@client-host:~# PGPASSWORD='<APP_PASSWORD>' psql -h 127.0.0.1 -p 6432 -U
appuser -d labdb_rw -c "BEGIN; INSERT INTO clients_lab(name, balance) VALUES
('FinalAfterRecovery', 4000); INSERT INTO operations_lab(client_id, amount,
operation_type) VALUES (currval('clients_lab_id_seq'), 4000,
'final_after_recovery'); COMMIT; SELECT * FROM clients_lab ORDER BY id DESC
LIMIT 5; SELECT * FROM operations_lab ORDER BY id DESC LIMIT 5;"
```

id	client_id	amount	operation_type	created_at
71	75	4000	final_after_recovery	2026-06-04 10:20:14.837415+00
39	43	2222	session_two_write	2026-06-04 10:14:41.968272+00
38	42	1111	session_one_write	2026-06-04 10:14:06.036186+00
37	38	3000	after_recovery	2026-06-04 10:05:57.186191+00
36	37	2000	after_failover	2026-06-04 09:57:00.42334+00

(5 rows)

```
root@client-host:~# ssh root@203.0.113.11 "sudo -u postgres psql -d labdb -c
\"SELECT * FROM clients_lab WHERE name = 'FinalAfterRecovery';\""
```

id	name	balance	created_at
75	FinalAfterRecovery	4000	2026-06-04 10:20:14.837415+00

(1 row)

На pg2 проверяем, что новые данные реплицировались:

```
ssh root@203.0.113.11
sudo -u postgres psql -d labdb -c "SELECT * FROM clients_lab;"
```


Архитектура

client / PgBouncer

pg1 primary

pg2 standby

В работе используется отдельная клиентская машина `client`. На ней запущены Ansible, PgBouncer и клиентские подключения `psql`. Узлы `pg1` и `pg2` используются только как серверы PostgreSQL.

PgBouncer aliases

labdb -> pg1

labdb_rw -> pg1

labdb_ro -> pg2

`labdb` и `labdb_rw` используются для подключения к текущему `primary`. `labdb_ro` используется для чтения с `standby`. После failover PgBouncer временно переключается на `pg2`, а после восстановления возвращается к исходной схеме.

Ansible templates

В проекте используются шаблоны:

- `postgresql-primary.conf.j2`;
- `postgresql-standby.conf.j2`;
- `pg_hba-primary.conf.j2`;
- `pg_hba-standby.conf.j2`;
- `pgbouncer.ini.j2`;
- `userlist.txt.j2`.

Через эти шаблоны Ansible задаёт параметры PostgreSQL, правила доступа, параметры PgBouncer и список пользователей PgBouncer.

Пользователи

В стенде используются два основных пользователя PostgreSQL:

- `replicator` — пользователь для потоковой репликации и выполнения `pg_basebackup`;
- `appuser` — пользователь приложения, через которого выполняются клиентские подключения к базе через PgBouncer.

Состояние базы до сбоя и после восстановления

До сбоя исходная схема была такой:

pg1: `pg_is_in_recovery = f`

pg2: `pg_is_in_recovery = t`

PgBouncer: `labdb/labdb_rw -> pg1, labdb_ro -> pg2`

После восстановления схема снова стала такой же:

```
pg1: pg_is_in_recovery = f
pg2: pg_is_in_recovery = t
PgBouncer: labdb/labdb_rw -> pg1, labdb_ro -> pg2
```

Финальная запись `FinalAfterRecovery` была выполнена через PgBouncer на `pg1` и затем найдена на `pg2`:

```
root@client-host:~# ssh root@203.0.113.11 "sudo -u postgres psql -d labdb -c
\"SELECT * FROM clients_lab WHERE name = 'FinalAfterRecovery';\""
 id |          name          | balance |          created_at
----+-----+-----+-----
 75 | FinalAfterRecovery |    4000 | 2026-06-04 10:20:14.837415+00
(1 row)
```

Это показывает, что после восстановления запись снова выполняется на `pg1`, а данные снова попадают на `pg2`, то есть исходная конфигурация восстановлена.

Фрагменты Ansible-шаблонов

В этом разделе приведены основные шаблоны, через которые Ansible настраивает PostgreSQL, правила доступа и PgBouncer.

Шаблон postgresql-primary.conf.j2

```
data_directory = '{{ pgdata }}'
hba_file = '{{ pg_config_dir }}/pg_hba.conf'
ident_file = '{{ pg_config_dir }}/pg_ident.conf'
external_pid_file = '/var/run/postgresql/{{ pg_version }}-main.pid'

listen_addresses = '*'
port = 5432

wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = 512MB
hot_standby = on
wal_log_hints = on

logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_connections = on
log_disconnections = on
log_duration = on
log_line_prefix = '%m [%p] %u@%d %r '
log_min_messages = info
log_min_error_statement = error

shared_buffers = 128MB
```

Шаблон postgresql-standby.conf.j2

```
data_directory = '{{ pgdata }}'
hba_file = '{{ pg_config_dir }}/pg_hba.conf'
ident_file = '{{ pg_config_dir }}/pg_ident.conf'
external_pid_file = '/var/run/postgresql/{{ pg_version }}-main.pid'

listen_addresses = '*'
port = 5432
hot_standby = on

logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_connections = on
log_disconnections = on
log_duration = on
```

```
log_line_prefix = '%m [%p] %u@%d %r '
log_min_messages = info
log_min_error_statement = error

shared_buffers = 128MB
```

Шаблон pg_hba-primary.conf.j2

```
local  all             postgres                                peer
local  all             all                                peer
host    all             all                127.0.0.1/32          scram-sha-256
host    all             all                ::1/128             scram-sha-256

host    {{ db_name }}   {{ app_user }}   {{ client_private_ip }}/32    scram-
sha-256

host    replication     {{ replication_user }} {{ pg2_private_ip }}/32    scram-
sha-256
```

Шаблон pg_hba-standby.conf.j2

```
local  all             postgres                                peer
local  all             all                                peer
host    all             all                127.0.0.1/32          scram-sha-256
host    all             all                ::1/128             scram-sha-256

host    {{ db_name }}   {{ app_user }}   {{ client_private_ip }}/32    scram-
sha-256

host    replication     {{ replication_user }} {{ pg1_private_ip }}/32    scram-
sha-256
```

Шаблон pgbouncer.ini.j2

```
[databases]
{{ db_name }} = host={{ pg1_private_ip }} port=5432 dbname={{ db_name }}
{{ db_name }}_rw = host={{ pg1_private_ip }} port=5432 dbname={{ db_name }}
{{ db_name }}_ro = host={{ pg2_private_ip }} port=5432 dbname={{ db_name }}

[pgbouncer]
listen_addr = {{ pgbouncer_listen_addr }}
listen_port = {{ pgbouncer_listen_port }}
auth_type = {{ pgbouncer_auth_type }}
auth_file = {{ pgbouncer_userlist }}
logfile = /var/log/postgresql/pgbouncer.log
pidfile = /var/run/postgresql/pgbouncer.pid
pool_mode = {{ pgbouncer_pool_mode }}
max_client_conn = 100
default_pool_size = 20
ignore_startup_parameters = extra_float_digits
admin_users = {{ app_user }}
```

```
log_connections = 1  
log_disconnections = 1
```

Шаблон userlist.txt.j2

```
"{{ app_user }}" "{{ app_password }}"
```

Скрипт проверки доступности и переключения

Скрипт работает на `client` по `systemd timer`. Он проверяет текущий `primary`, при отказе повышает `standby` до `primary` и переключает `PgBouncer`.

```
#!/usr/bin/env bash
set -u

PRIMARY_IP="{{ pg1_private_ip }}"
STANDBY_IP="{{ pg2_private_ip }}"
SSH_USER="{{ failover_ssh_user }}"
DB_NAME="{{ db_name }}"
DB_USER="{{ app_user }}"
DB_PORT="5432"
PGDATA="{{ pgdata }}"
PG_CTL="{{ pg_ctl }}"
PGBOUNCER_CONFIG="{{ pgbouncer_config }}"
LOG_FILE="{{ failover_log_file }}"
LOCK_FILE="/var/run/pg_auto_failover.lock"
STATE_FILE="{{ failover_state_file }}"

mkdir -p "$(dirname "$STATE_FILE")"
touch "$LOG_FILE"

exec 9>"$LOCK_FILE"
if ! flock -n 9; then
    exit 0
fi

log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') $*' >> "$LOG_FILE"
}

CURRENT_PRIMARY="$(cat "$STATE_FILE" 2>/dev/null || echo "$PRIMARY_IP")"

if [ "$CURRENT_PRIMARY" = "$STANDBY_IP" ]; then
    log "Текущий primary уже $STANDBY_IP. Действие не требуется."
    exit 0
fi

log "Проверяем основной узел $PRIMARY_IP..."

if pg_isready -h "$PRIMARY_IP" -p "$DB_PORT" -d "$DB_NAME" -q; then
    log "Основной узел $PRIMARY_IP доступен."
    echo "$PRIMARY_IP" > "$STATE_FILE"
    exit 0
fi

log "Основной узел $PRIMARY_IP недоступен. Проверяем резервный узел $STANDBY_IP..."
```

```

if ! pg_isready -h "$STANDBY_IP" -p "$DB_PORT" -q; then
    log "Резервный узел $STANDBY_IP тоже недоступен. Прерываем выполнение."
    exit 1
fi

log "Резервный узел $STANDBY_IP доступен. Выполняем promote через SSH..."
ssh -o BatchMode=yes -o ConnectTimeout=5 "$SSH_USER@$STANDBY_IP" "sudo -u
postgres $PG_CTL promote -D $PGDATA" >> "$LOG_FILE" 2>&1
PROMOTE_RC=$?
if [ "$PROMOTE_RC" -ne 0 ]; then
    log "Команда promote вернула rc=$PROMOTE_RC. Возможно, узел уже повышен; всё
равно проверяем состояние."
fi

sleep 5
RECOVERY_STATUS="$(psql -h "$STANDBY_IP" -p "$DB_PORT" -U "$DB_USER" -d
"$DB_NAME" -tAc "SELECT pg_is_in_recovery();" 2>>"$LOG_FILE" | tr -d
'[:space:]')"

if [ "$RECOVERY_STATUS" != "f" ]; then
    log "Promote не выполнен или не завершён. pg_is_in_recovery=$RECOVERY_STATUS"
    exit 1
fi

log "Резервный узел $STANDBY_IP успешно повышен до primary. Переключаем
PgBouncer."
sed -i "s/host=$PRIMARY_IP/host=$STANDBY_IP/g" "$PGBOUNCER_CONFIG"
systemctl restart pgbouncer

echo "$STANDBY_IP" > "$STATE_FILE"
log "PgBouncer переключён на $STANDBY_IP и перезапущен."

```

Systemd service и timer

```

[Unit]
Description=Лабораторная работа 3: проверка доступности PostgreSQL и PgBouncer
After=network-online.target pgbouncer.service

[Service]
Type=oneshot
ExecStart=/usr/local/bin/pg_auto_failover.sh

[Unit]
Description=Лабораторная работа 3: периодическая проверка доступности PostgreSQL

[Timer]
OnBootSec=10
OnUnitActiveSec={{ failover_check_interval_seconds }}
AccuracySec=1

```

```
[Install]  
WantedBy=timers.target
```


Вывод

В ходе лабораторной работы был развёрнут отказоустойчивый стенд PostgreSQL из двух узлов и отдельной клиентской машины с PgBouncer. Была настроена потоковая репликация между `pg1` и `pg2`, создана тестовая база данных и продемонстрирована работа нескольких клиентских сессий.

В процессе симуляции сбоя раздел с `PGDATA` на основном узле был заполнен мусорными файлами. После появления ошибок записи был выполнен автоматический failover: резервный сервер `pg2` стал новым `primary`, а PgBouncer был переключён на него. Затем исходный основной узел был восстановлен, актуализирован с помощью `pg_basebackup`, после чего роли были возвращены к исходной конфигурации `pg1 primary` -> `pg2 standby`.